
COMP0037 COURSEWORK 1

A PREPRINT

Group Name: Group 1
Department of Computer Science
University College London
London, WC1E 6BT

March 3, 2025

Part 1:

Q1(a)

Breadth-First Search (BFS) follows a First-In-First-Out (FIFO) queue structure, systematically managing explored nodes. In LaValle's discrete forward planner pseudocode, the *Insert* function appends new elements to the tail of the queue, while *GetFirst* retrieves elements from the head. This method ensures that the search expands layer by layer, systematically exploring all reachable nodes. As a result, BFS guarantees finding the shortest path in an unweighted graph. However, BFS requires storing all nodes at the current depth before expanding further, making it memory-intensive and potentially inefficient when applied to large state spaces. Algorithm 1 provides the pseudocode implementation of BFS in LaValle's Forward Search framework.

Algorithm 1 Breadth-First Search (BFS) in LaValle's Forward Search Framework

```
1: function FORWARDSEARCH( $x_I, X_G$ )
2:   Initialize an empty queue  $Q$ 
3:    $Q.Insert(x_I)$  and mark  $x_I$  as alive
4:   while  $Q$  is not empty do
5:      $x \leftarrow Q.GetFirst()$  ▷ Retrieve the front of the queue (FIFO)
6:     if  $x \in X_G$  then
7:       return SUCCESS
8:     end if
9:     for each  $u \in U(x)$  do
10:       $x' \leftarrow f(x, u)$ 
11:      if  $x'$  is unvisited then
12:        Mark  $x'$  as alive and set parent as  $x$ 
13:         $Q.Insert(x')$  ▷ Insert at the end of the queue
14:      else
15:        Resolve duplicate  $x'$ 
16:      end if
17:    end for
18:    Mark  $x$  as dead
19:  end while
20:  return FAILURE
21: end function
```

Depth-First Search (DFS) follows a Last-In-First-Out (LIFO) approach, using a stack to manage explored nodes. In LaValle's discrete forward planner pseudocode, the *Insert* function appends new elements to the top of the stack, while *GetFirst* retrieves elements from the same position. This method allows the search to proceed deeply into the search space before backtracking, making DFS suitable for traversing large graphs with lower memory consumption compared

to BFS. However, unlike BFS, DFS does not guarantee finding the shortest path and may become trapped in deep or infinite search paths, particularly in cyclic or large search spaces. Algorithm 2 presents the DFS pseudocode as implemented in LaValle’s Forward Search framework.

Algorithm 2 Depth-First Search (DFS) in LaValle’s Forward Search Framework

```

1: function FORWARDSEARCH( $x_I, X_G$ )
2:   Initialize an empty stack  $Q$ 
3:    $Q.Insert(x_I)$  and mark  $x_I$  as alive
4:   while  $Q$  is not empty do
5:      $x \leftarrow Q.GetLast()$  ▷ Retrieve the last inserted element (LIFO)
6:     if  $x \in X_G$  then
7:       return SUCCESS
8:     end if
9:     for each  $u \in U(x)$  do
10:       $x' \leftarrow f(x, u)$ 
11:      if  $x'$  is unvisited then
12:        Mark  $x'$  as alive and set parent as  $x$ 
13:         $Q.Insert(x')$  ▷ Insert at the top of the stack
14:      else
15:        Resolve duplicate  $x'$ 
16:      end if
17:    end for
18:    Mark  $x$  as dead
19:  end while
20:  return FAILURE
21: end function

```

Table 1 summarizes the advantages and disadvantages of BFS and DFS. BFS ensures that the shortest path is found but requires significantly more memory due to its level-wise expansion. DFS, by contrast, requires less memory but is prone to inefficiencies, as it may explore deep paths unnecessarily and fail to find the optimal solution.

Algorithm	Advantages	Disadvantages
BFS	- Explores all reachable nodes. - Guarantees the shortest path in an un-weighted graph.	- Memory-intensive, as it must store multiple nodes in the queue. - Can be slow and inefficient in large state spaces.
DFS	- Uses less memory, as it only tracks the depth of the path. - Can be faster if the goal is located deep in the search space.	- May get stuck in deep branches, leading to inefficiency. - Does not guarantee the shortest path.

Table 1: Comparison of BFS and DFS in LaValle’s Forward Search Framework

Q1(b)

To evaluate the performance of Breadth-First Search (BFS) and Depth-First Search (DFS) in the airport scenario, we adopt a two-fold approach. First, we conduct a quantitative analysis by examining the number of cells visited and the path cost associated with each search. Second, we perform a qualitative assessment by analyzing the generated paths and identifying any unexpected behaviors using a visualizer.

DFS is expected to visit fewer cells since it explores deep into the search space before backtracking. However, experimental results reveal an unexpected trend: DFS visits significantly more cells on average than BFS. As shown in Table 2, DFS visits an average of 1242 cells, significantly exceeding BFS’s 502. This result suggests that DFS frequently becomes trapped in deep branches, leading to exhaustive exploration of the search space and suboptimal path costs. In contrast, BFS systematically explores all possible paths layer by layer, resulting in a much lower average path cost (22 for BFS vs. 290 for DFS), demonstrating its efficiency in finding more optimal routes.

Algorithm	Average Path Cost	Average Cells Visited
BFS	22	502
DFS	290	1242

Table 2: Comparison of Average Path Costs and Cells Visited for BFS and DFS

In most cases, DFS fails to generate an optimal path and instead becomes trapped in inefficient search loops. Figure 1 and 2 illustrate Bin 13 search behaviour. BFS efficiently finds an optimal path, visiting only a quarter of the search space. On the other hand, DFS explores excessively before settling on a suboptimal path, demonstrating its inefficiency.

While DFS typically performs poorly, there are exceptions. In certain cases, DFS can generate paths with costs comparable to BFS. Figure 1 and 2 show the search for Bin 7 using both BFS and DFS, representing one of the few cases where DFS produces an optimal path. Although the paths differ slightly, their path costs are nearly identical in this unweighted scenario. In such cases, DFS visits fewer cells than BFS, making it a faster and more memory-efficient alternative. For Bin 7, DFS visited only 184 cells compared to BFS's 1734, yet the path cost remained close (63.84 for DFS vs. 63.25 for BFS). In this cases, DFS shows potential memory efficiency and speed advantages, in return to optimality.

Figure 3 and 4 show the search for Bin 7 using both BFS and DFS, representing one of the few cases where DFS produces an optimal path. Although the paths differ slightly, their path costs are nearly identical in this unweighted scenario. In such cases, DFS visits fewer cells than BFS, making it a faster and more memory-efficient alternative.

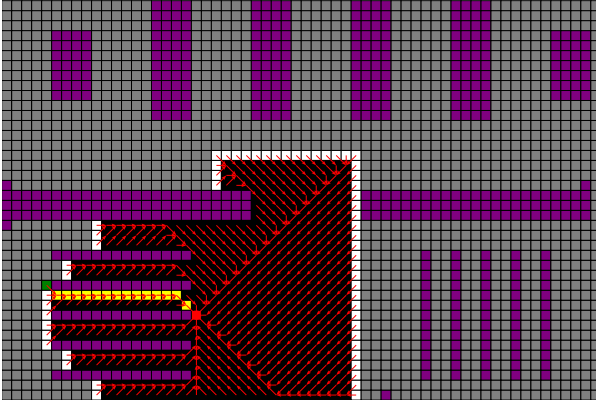


Figure 1: Bin 13: BFS

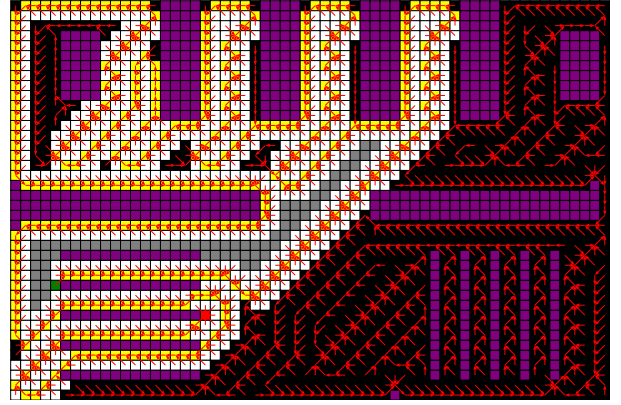


Figure 2: Bin 13: DFS

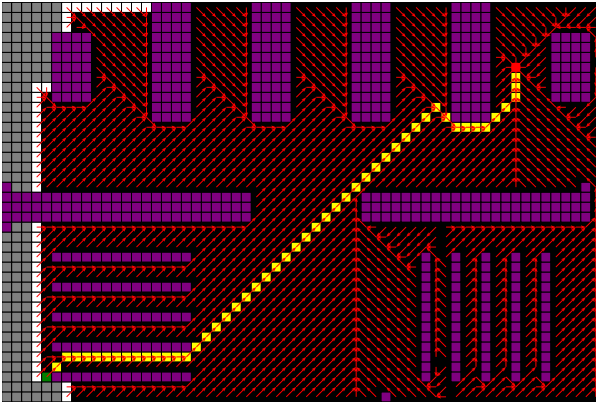


Figure 3: Bin 07: BFS

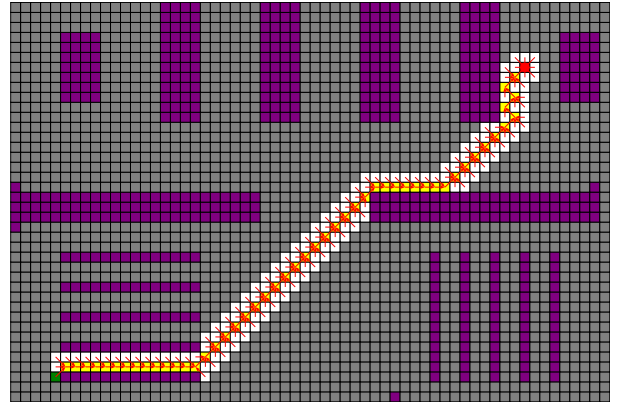


Figure 4: Bin 07: DFS

This analysis confirms that BFS consistently outperforms DFS in terms of reliability and efficiency, particularly in scenarios where exhaustive exploration is unnecessary. While DFS occasionally demonstrates computational advantages by exploring fewer nodes, its lack of systematic exploration often results in suboptimal paths and excessive node expansion. Consequently, BFS remains the preferred method for ensuring high-quality pathfinding in structured environments.

Q1(c)

Dijkstra's algorithm operates similarly to Breadth-First Search (BFS) but introduces a key modification: it prioritizes exploration based on the lowest path cost rather than uniformly expanding layer by layer. Instead of using a First-In-First-Out (FIFO) queue like BFS, Dijkstra's algorithm employs a priority queue, where the priority is determined by the total cost from the initial state to the given cell.

In LaValle's discrete forward planner pseudocode, the key modifications compared to BFS include:

- The *Insert* function places new nodes into the priority queue, ensuring that nodes closest to the initial state are explored first.
- The *GetFirst* function extracts the node with the lowest cost from the priority queue, rather than retrieving the nodes in the order they were inserted.
- Instead of expanding equally in layers, Dijkstra's algorithm selectively expands nodes closer to the initial state first, leading to a more efficient exploration in weighted environments.

Dijkstra's algorithm guarantees finding the shortest path in a weighted graph, making it more suitable than BFS for environments with non-uniform travel costs. It also computes the optimal path to all cells it encounters during the search process. However, this advantage comes at the cost of increased computational overhead due to sorting operations when managing the priority queue. Algorithm 3 presents the implementation of the Dijkstra algorithm in the LaValle Forward Search framework.

Algorithm 3 Dijkstra's Algorithm in LaValle's Forward Search Framework

```

1: function FORWARDSEARCH( $x_I, X_G$ )
2:   Initialize an empty **priority queue**  $Q$ 
3:    $Q.Insert(x_I, 0)$  and mark  $x_I$  as alive ▷ Start node with cost 0
4:   while  $Q$  is not empty do
5:      $x \leftarrow Q.GetFirst()$  ▷ Retrieve the node with the lowest cost
6:     if  $x \in X_G$  then
7:       return SUCCESS
8:     end if
9:     for each  $u \in U(x)$  do
10:       $x' \leftarrow f(x, u)$ 
11:       $cost' \leftarrow$  cost to reach  $x'$  from  $x$ 
12:      if  $x'$  is unvisited or  $cost'$  is lower than previous cost then
13:        Update cost of  $x'$ 
14:        Mark  $x'$  as alive and set parent as  $x$ 
15:         $Q.Insert(x', cost')$  ▷ Insert based on minimum cost
16:      else
17:        Resolve duplicate  $x'$ 
18:      end if
19:    end for
20:    Mark  $x$  as dead
21:  end while
22:  return FAILURE
23: end function

```

Q1(d)

Dijkstra's algorithm expands nodes based on the lowest cumulative path cost, ensuring an optimal solution in an environment with uniform or varying movement costs. The implementation follows a similar structure to the BreadthFirstPlanner but incorporates a priority queue for cost-based expansion.

```
class DijkstraPlanner(PlannerBase):
    def __init__(self, occupancy_grid: OccupancyGrid):
        PlannerBase.__init__(self, occupancy_grid)
        self.priority_queue = PriorityQueue()
        self.counter = 0 # Used to handle cells with the same path cost
```

In this implementation, a PriorityQueue is used to prioritize cells with lower path costs. The additional counter ensures that if multiple cells have the same cost, they are processed in a consistent order.

The push_cell_onto_queue() function inserts a cell into the queue with its cumulative path cost:

```
def push_cell_onto_queue(self, cell):
    self.counter += 1
    self.priority_queue.put((cell.path_cost, self.counter, cell))
```

The pop_cell_from_queue() function retrieves the cell with the lowest path cost. To prevent processing outdated entries, the function verifies that the retrieved cost matches the cell's current path cost:

```
def pop_cell_from_queue(self):
    while not self.priority_queue.empty():
        cost, _, cell = self.priority_queue.get()
        if cost > cell.path_cost:
            continue
        return cell
    return None
```

To ensure accurate path reconstruction, the mark_cell_as_visited_and_record_parent() function updates a cell's status and assigns its parent:

```
def mark_cell_as_visited_and_record_parent(self, cell, parent_cell: Optional[SearchGridCell]):
    cell.set_label(SearchGridCellLabel.ALIVE)
    cell.set_parent(parent_cell)

    if parent_cell is not None:
        cell.path_cost = parent_cell.path_cost + self.compute_l_stage_additive_cost(parent_cell, cell)
```

When encountering a duplicate cell in the queue, the resolve_duplicate() function determines whether the new path is more efficient. If the new cost is lower, the function updates the cell's path cost and re-enqueues it:

```
def resolve_duplicate(self, cell, parent_cell):
    new_cost = parent_cell.path_cost + self.compute_l_stage_additive_cost(parent_cell, cell)
    if new_cost < cell.path_cost:
        cell.set_label(SearchGridCellLabel.ALIVE)
        cell.path_cost = new_cost
        cell.set_parent(parent_cell)
        self.push_cell_onto_queue(cell)
```

These modifications ensure that Dijkstra's algorithm systematically expands the lowest-cost nodes first, guaranteeing optimal paths while effectively handling duplicate entries and dynamic cost updates.

Q1(e)

Dijkstra’s algorithm performs similarly to BFS in terms of the number of visited cells while significantly outperforming DFS. The key distinction between BFS and Dijkstra lies in Dijkstra’s cost-awareness, which allows it to prioritize lower-cost paths using a priority queue. As shown in Table 3, Dijkstra visits slightly more cells than BFS, with an average of 504 nodes explored compared to BFS’s 502. This slight increase in search space is due to the algorithm’s priority-based expansion, ensuring that lower-cost paths are explored first. However, this leads to an improvement in path cost, with Dijkstra achieving an average path cost of 21 compared to BFS’s 22, demonstrating its ability to optimize routes dynamically based on cost.

Algorithm	Average Path Cost	Average Cells Visited
BFS	22	502
DFS	290	1242
Dijkstra	21	504

Table 3: Comparison of Average Path Costs and Cells Visited for BFS, DFS, and Dijkstra

Although the paths generated by BFS and Dijkstra are often similar, Dijkstra consistently optimizes path cost by prioritizing lower-cost expansions. Figure 5 and 6 provide a visual comparison of the paths generated for Bin 01 using BFS and Dijkstra. Unlike BFS, which expands nodes uniformly, Dijkstra selectively avoids unnecessary diagonal movements when they are costlier, while still utilizing them when beneficial. This cost-aware decision-making ensures that Dijkstra consistently maintains an optimal path while balancing efficiency.

To conclude, Dijkstra effectively integrates BFS’s systematic exploration with cost-awareness, resulting in more efficient movement selection and slightly lower path costs. While BFS remains a strong choice in uniform-cost environments, Dijkstra demonstrates advantages in scenarios where movement costs vary, such as those incorporating higher-cost diagonal moves. This makes Dijkstra the preferred choice for optimal path planning when cost efficiency is a priority. The following section, Q1(f), will further analyze Dijkstra’s performance in an environment where varying path costs influence route selection.

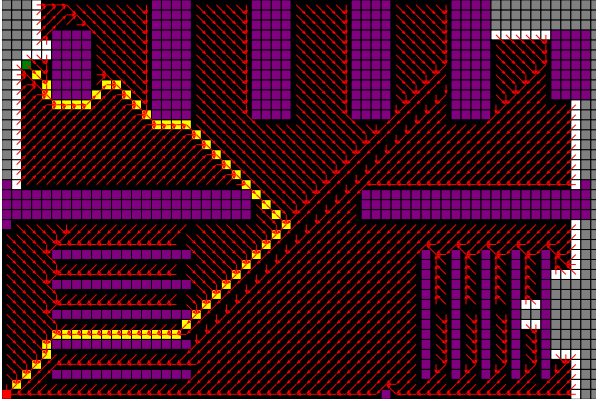


Figure 5: Bin 01: BFS

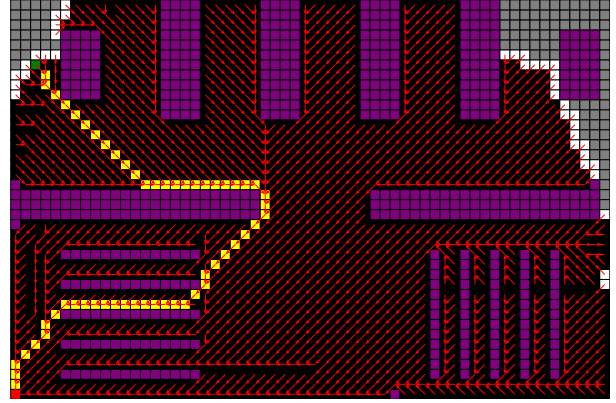


Figure 6: Bin 01: Dijkstra

Q1(f)

The function `compute_transition_cost()` was already well-implemented in the source code. The only modification made was adding an `else` condition to ensure the base cost is returned correctly when the cell is neither a secret door nor a customs area.

Enabling cell-type traversability costs leads to adjustments in the Dijkstra planner’s behaviour. In cases where the original path does not pass through restricted areas, the computed paths and statistics remain unchanged. However, in environments where traversal costs influence decision-making, notable deviations are observed. Specifically, for Bin 01 and Bin 07, Dijkstra modifies its planned routes to avoid high-cost zones.

Table 4 presents the performance comparison of the Dijkstra algorithm with and without traversal cost adjustments. Without transition penalties, the path costs for Bin 01 and Bin 07 are 66 and 62, respectively. However, after

incorporating cell-specific penalties, the path costs increase significantly to 133 for Bin 01 and 80 for Bin 07, reflecting the impact of avoiding high-cost areas. Despite this, the number of visited cells remains relatively stable, increasing only slightly from 1766 to 1844 for Bin 01 and from 1845 to 1836 for Bin 07, showing that Dijkstra efficiently reroutes around costly custom area without excessive search expansion.

Algorithm	Path Costs	Cells Visited
Dijkstra bin 01	66	1766
Dijkstra bin 07	62	1845
Dijkstra Transition Cost bin 01	133	1844
Dijkstra Transition Cost bin 07	80	1836

Table 4: Performance of Dijkstra Algorithm with and without cell-type traversability costs

The impact of these adjustments is further visualized in Figure 7 and 8, where the updated Dijkstra paths for Bin 01 and Bin 07 are displayed. The modified routes actively avoid the customs area, which imposes a significant path cost of 100. Instead, the planner selects a slightly longer but more cost-effective path by utilizing secret doors along the scenario’s right edge. While this increases the total path cost compared to the original Dijkstra route, it remains substantially lower than traversing the customs area, demonstrating the algorithm’s ability to dynamically optimize paths based on cost constraints.

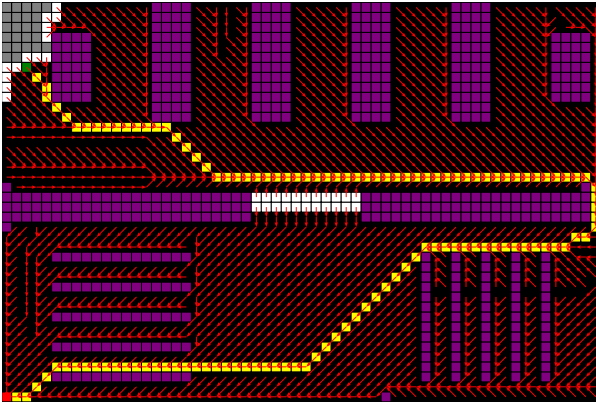


Figure 7: Bin 01: Dijkstra with Transition Cost

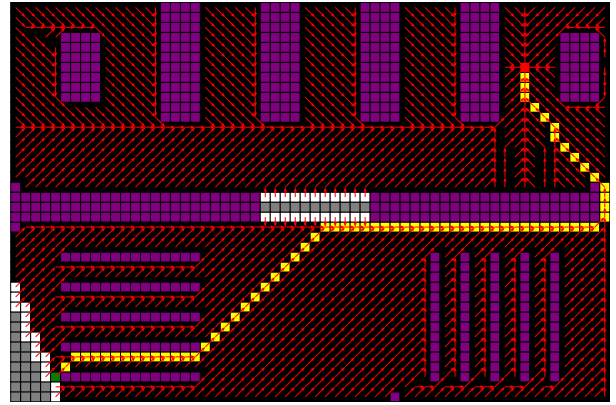


Figure 8: Bin 07: Dijkstra with Transition Cost

These results illustrate how the introduction of traversal cost penalties influences Dijkstra’s decision-making process, leading to more practical and efficient routing in complex environments. The ability to dynamically adjust paths based on cost constraints ensures that the planner avoids high-cost zones while maintaining reasonable computational efficiency.

Q2(a)

A* search extends Dijkstra’s algorithm by incorporating greedy approaches to reduce the amount of search, allowing it to guide the search more efficiently while guarantees a optimal path. A* search based on a priority queue in order of the shortest predicted total path cost with assist of heuristic function.

In LaValle’s discrete forward planner pseudocode, A* introduces the following modifications:

- The *Insert* function places new nodes into a priority queue, similar to Dijkstra’s algorithm. However, A* uses a combined cost function:

$$f(x) = g(x) + h(x)$$

where:

- $g(x)$ is the actual cost from the start node to x .
- $h(x)$ is the heuristic estimate of the cost from x to the goal.
- The *GetFirst* function extracts the node with the lowest total estimated cost $f(x)$ from the priority queue.

- A* selectively expands nodes that appear to be on the best path to the goal, rather than just the cheapest (Dijkstra) or the closest (Greedy Best-First Search).

By incorporating a heuristic function, A* significantly reduces the number of nodes explored, making it faster than Dijkstra's algorithm in many cases. However, its performance heavily depends on the quality of the heuristic—an inadmissible or inconsistent heuristic may lead to suboptimal solutions or inefficiency.

Algorithm 4 presents the implementation of A* search in LaValle's Forward Search framework.

Algorithm 4 A* Algorithm in LaValle's Forward Search Framework

```

1: function FORWARDSEARCH( $x_I, X_G$ )
2:   Initialize an empty priority queue  $Q$ 
3:    $Q.Insert(x_I, 0 + h(x_I))$  and mark  $x_I$  as alive ▷ Start node with heuristic
4:   while  $Q$  is not empty do
5:      $x \leftarrow Q.GetFirst()$  ▷ Retrieve the node with the lowest estimated cost
6:     if  $x \in X_G$  then
7:       return SUCCESS
8:     end if
9:     for each  $u \in U(x)$  do
10:       $x' \leftarrow f(x, u)$ 
11:       $g' \leftarrow$  cost to reach  $x'$  from  $x$ 
12:       $f' \leftarrow g' + h(x')$  ▷ Total estimated cost
13:      if  $x'$  is unvisited or  $f'$  is lower than previous cost then
14:        Update cost of  $x'$ 
15:        Mark  $x'$  as alive and set parent as  $x$ 
16:         $Q.Insert(x', f')$  ▷ Insert based on heuristic-guided cost
17:      else
18:        Resolve duplicate  $x'$ 
19:      end if
20:    end for
21:    Mark  $x$  as dead
22:  end while
23:  return FAILURE
24: end function

```

Q2(b)

A heuristic function $h(x)$ is admissible if it is always non-negative and does not overestimate the actual cost to the goal. This ensures that the A* algorithm always finds the optimal solution.

To be admissible, the heuristic function must satisfy the following condition for all states s_t :

$$0 \leq \hat{G}_t(s_t; \pi_T) \leq G_t(s_t; \pi_T) \quad (1)$$

where:

- $\hat{G}_t(s_t; \pi_T)$ represents the estimated cost using the heuristic function.
- $G_t(s_t; \pi_T)$ represents the actual cost of s_t to the goal under the optimal policy π_T .

This means that the heuristic function should never overestimate the optimal cost (G_t), ensuring that the A* search remains optimal and efficient.

An example of admissible heuristic function is the Zero heuristic defined as:

$$h(x) = 0 \quad (2)$$

This heuristic assumes that the predict cost is always 0. As it is non-negative and never overestimate the path cost, it is always admissible. However, this heuristic is meaningless as the combined cost will equals to the actual cost, which makes our A* algorithm identical to Dijkstra.

An example of inadmissible heuristic function is the Squared Euclidean distance is defined as:

$$h(x) = (x_{\text{goal}} - x_{\text{current}})^2 + (y_{\text{goal}} - y_{\text{current}})^2 \quad (3)$$

Since the squared distance always overestimates the true path cost, it violates admissibility and can lead A* to an inefficient or incorrect solution.

Manhattan distance is also an example of inadmissible heuristic, defined as:

$$h(x) = |x_{\text{goal}} - x_{\text{current}}| + |y_{\text{goal}} - y_{\text{current}}| \quad (4)$$

This heuristic is inadmissible in diagonal movement scenarios since it can underestimate the cost when diagonal moves are allowed in unweighted graphs. For example, when the object moves from (0,0) to (3,3), the exact path cost is 3, but Manhattan distance gives an estimation of $3 + 3 = 6$, which overestimated the path cost. However, in a strictly grid-based environment with only horizontal and vertical moves or a weighted environment, the Manhattan distance can still be admissible and guarantee an optimal solution.

Q2(c)

The best possible heuristic for A* search is a heuristic $h^*(x)$ which always returns the exact cost of the shortest path from the current state to the goal. Since $h^*(x)$ exactly represents the true cost-to-go, A* will always expand the optimal path without any unnecessary node exploration, achieving perfect efficiency as well as optimal path.

While computing $h^*(x)$ would ensure optimal pathfinding, it is generally impractical because determining $h^*(x)$ requires solving the shortest path problem in advance. There is no point of implementing a search algorithm as the heuristic itself would already contain the solution. Instead, practical heuristics such as Euclidean distance, Manhattan distance, or domain-specific approximations are used to balance computational efficiency and search effectiveness. These heuristics provide a reasonable estimate of the remaining cost while avoiding the prohibitive overhead of computing $h^*(x)$ exactly.

Q2(d)

The primary distinction between A* and Dijkstra's algorithm is the integration of a heuristic function to guide the search. In our implementation, this heuristic is computed in the `heuristic()` method, which calculates the Euclidean distance between the current cell and the goal:

```
def heuristic(self, cell):
    goal_x, goal_y = self.goal.coords()
    cell_x, cell_y = cell.coords()
    return math.sqrt((goal_x - cell_x) ** 2 + (goal_y - cell_y) ** 2)
```

The heuristic is then implemented into the priority function of A* in `push_cell_onto_queue()`. Instead of relying solely on the actual path cost, as in Dijkstra's algorithm, A* computes a combined cost:

```
def push_cell_onto_queue(self, cell):
    if cell is None:
        return
    self.counter += 1
    f_cost = cell.path_cost + self.heuristic(cell)
    self.priority_queue.put((f_cost, self.counter, cell))
```

Here, the total cost function $f(n) = g(n) + h(n)$ is used, where $g(n)$ represents the known path cost from the start node and $h(n)$ is the heuristic estimate. The priority queue sorts nodes based on this combined value, ensuring that cells that are both closer to the goal and have lower known costs are expanded first.

Similarly, `pop_cell_from_queue()` retrieves the node with the lowest $f(n)$ from the priority queue:

```
def pop_cell_from_queue(self):
    while not self.priority_queue.empty():
        f_cost, _, cell = self.priority_queue.get()
        current_f = cell.path_cost + self.heuristic(cell)
        if f_cost > current_f:
            continue
        return cell
```

To prevent outdated entries from being processed, the function recalculates the current $f(n)$ value and compares it with the extracted cost from the queue. If an outdated node is found, it is discarded.

All other core functionalities remain inherited from `DijkstraPlanner`, ensuring that A* retains the fundamental path-searching logic while improving efficiency through informed decision-making based on heuristics.

Q2(e)

Table 5 presents the average path costs and cells visited for each algorithm. A* demonstrates a strong balance between efficiency and path cost, visiting significantly fewer nodes (192 on average) while maintaining a near-optimal path cost (24). In comparison, BFS and Dijkstra visit over 500 nodes on average, with BFS failing to account for movement cost and Dijkstra expanding many unnecessary nodes in pursuit of the absolute lowest-cost path. DFS performs the worst, expanding more than 1200 nodes and generating extremely high-cost paths due to its inefficient exploration strategy.

Algorithm	Average Path Cost	Average Cells Visited
BFS	22	502
DFS	290	1242
Dijkstra	21	504
Dijkstra (Penalty)	24	468
A*	24	192

Table 5: Comparison of Average Path Costs and Cells Visited for All Algorithms

A key advantage of A* over Dijkstra is its use of heuristics to guide the search. While Dijkstra strictly expands based on the lowest cumulative cost, A* prioritizes states that are both cost-efficient and closer to the goal. This results in significantly fewer nodes being expanded while still achieving a path cost close to Dijkstra's. The difference becomes more evident when analysing environments with varying traversal costs. Unlike BFS and DFS, which fail to account for cost variations and often produce inefficient paths, A* adapts dynamically to different cost structures.

Figure 9 and 10 visualise the paths generated by A*.

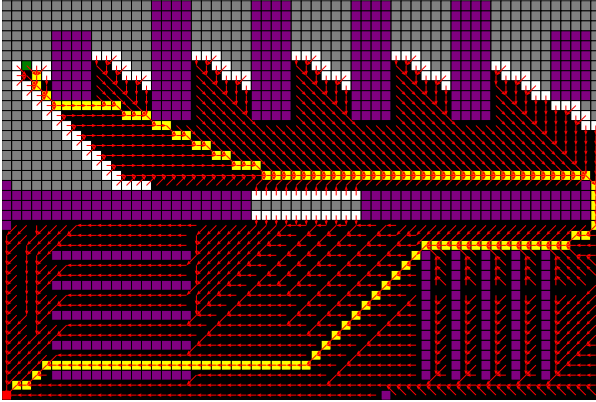


Figure 9: A* Path for Bin 01

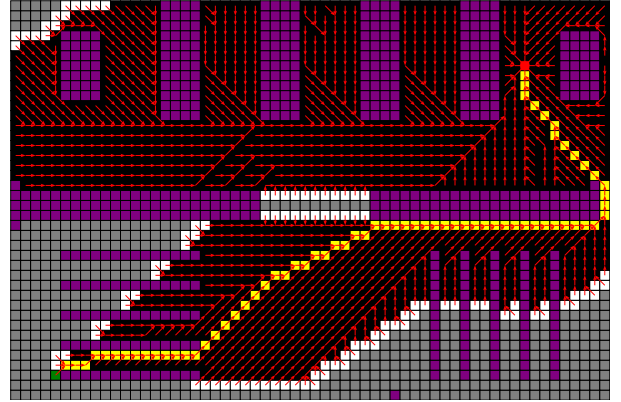


Figure 10: A* Path for Bin 07

The introduction of different cell traversal costs further highlights A*'s efficiency. When certain regions impose movement penalties, BFS and DFS remain unaffected as they do not incorporate cost into their decision-making. This results in frequent entry into high-cost zones. Dijkstra adapts by avoiding penalized areas but does so at the cost of significantly increased node expansions. A*, however, efficiently balances path cost with search efficiency, selectively avoiding high-penalty regions while maintaining a focused search toward the goal. This behaviour is particularly evident in the penalty-adjusted Dijkstra experiment, where the modified cost function influenced the algorithm's expansion strategy. While this improved cost-awareness, it still resulted in more expansions than A*, reinforcing the latter's advantage in balancing efficiency and cost optimization.

This comparison demonstrates that A* achieves the best trade-off between computational efficiency and cost-aware pathfinding. It consistently finds paths close to the lowest possible cost while expanding significantly fewer nodes

compared to cost-agnostic algorithms and even Dijkstra. The ability to incorporate traversal costs without excessive expansion makes A* the most effective approach in environments with varying terrain penalties.

Part 2:

Q3(a)

Finite Markov Decision Processes (FMDP) is a mathematical model used to handling uncertainty in Robotics. In this scenario, the environment is fixed (the airport). So, the only uncertainty is the randomness in the robot movement.

A **FMDP** consists of: $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:

- **Finite set of states $\mathcal{S}$:** This represents all possible locations where the robot can be within the environment (airport). It includes all the discrete states of the robots $S_t \in \mathcal{S}$ which are represented as a discrete grid coordinate: $S_t = \{(x, y) \mid x \in X, y \in Y\}$
- **Finite set of actions $\mathcal{A}$:** This consists of all possible moves the robot can take from any given state (at any position) $A_t \in \mathcal{A}(s)$. In a typical grid-based environment, the robot may have the following actions:
 - Cardinal Directions (Up, Down, Left, Right)
 - Diagonal Directions
 - Special pathways (e.g., a secret door)
- **State Transition Probabilities $\mathcal{P}$:** Due to the randomness in the robot movement, the state transition probability is not deterministic. The probability of transitioning from state s_{t-1} (initial position) to state s_t (next position) after taking action a_{t-1} (one movement) can be written as:

$$p(s_t \mid s_{t-1}, a_{t-1}) = \begin{cases} p, & \text{if the robot moves as intended,} \\ 0.5(1-p), & \text{if the robot deviates left,} \\ 0.5(1-p), & \text{if the robot deviates right.} \end{cases} \quad (5)$$

- **Reward Function $\mathcal{R}$:** This consists of all the possible reward $r_t \in \mathcal{R}$, and determines how desirable each state-action pair is. It ensures that the robot follows an optimal path while avoiding high-cost regions. In this scenario we can use traversability costs to design the reward function. Possible reward design:
 - **Reaching the goal state:** Large positive reward (+50)
 - **Moving one step:** Small negative reward to encourage efficiency (-1)
 - **Doing nothing:** Small negative reward to encourage moving (-1)
 - **Colliding with an obstacle:** small negative reward depends on the obstacle (-1 to -10)
 - **Entering a high-cost region (customs area):** Heavy penalty (-100)
 - **Using a secret door:** Moderate penalty (-5)
- **Discount factor γ :** This determines the extent to which future rewards influence the robot's decision-making. In this scenario, the robot must take a large number of actions, making it crucial to focus on optimizing the entire path to the goal rather than simply maximizing immediate rewards. To achieve this, we accumulate all rewards from the current time step until the end of the task and apply a $\gamma \in [0, 1]$ to each subsequent reward. This approach results in G_t , ensuring that future rewards are considered while still prioritizing nearer-term outcomes:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (6)$$

- **If $\gamma \approx 1$:** The robot focuses on long-term rewards, encouraging strategic planning.
- **If $\gamma \approx 0$:** The robot is short-sighted, focusing on immediate gains.

Q3(b)

From the previous question, we model the system as a **Finite Markov Decision Process (FMDP)**.

- To evaluate and improve policies, we define the **State Value Function** which defines the expected cumulative reward when starting from state s_t and following policy π :

$$v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] \quad (7)$$

- Computed recursively using the **Bellman Equation**:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_\pi(s')] \quad (8)$$

- In Markov Decision Processes (MDPs), the **discount factor** γ determines the importance of future rewards:
 - If $\gamma = 1$, future rewards are not discounted, meaning they are equally weighted as immediate rewards. In non-terminal states, the value function may accumulate rewards indefinitely, leading to divergence:
 - If $\gamma < 1$, future rewards decrease exponentially over time, ensuring that the value function converges.
- From `low_level_policy.py` we know the initial policy is always moving rightward:

```

1 type_creator = lambda x, y:
2   LowLevelActionType.TERMINATE if airport_map.cell(x, y).is_terminal()
3   else LowLevelActionType.NONE if airport_map.cell(x, y).is_obstruction()
4   else LowLevelActionType.RIGHT

```

Listing 1: The initial policy

- But from the given airport map we can find that no position can reach the termination with this initial policy without any policy improvement. The robot will walk rightward until it hits an obstacle, then return to its original position and continue to hit the obstacle again (Shown in Figure 5). This will be in an infinite loop. This is a **non-episodic task**.
- Since $\gamma = 1$, each iteration adds rewards from future states indefinitely. This leads to an ever-increasing value function, making the algorithm diverge instead of converging.
- The new **value function** is computed as:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + v_\pi(s')] \quad (9)$$

A straightforward solution is to modify the **discount factor**, such as setting: $\gamma = 0.9$. (But actually this isn't a good discount factor, the reason will show in the next few questions) This ensures that future rewards gradually reduce, allowing the value function to converge.

- In code, we can solve the problem by using the method `set_gamma()` in the class `PolicyIterator()` (which inherit the parent class `DynamicProgrammingBase()`). The result is shown in Figure 6.

```

1 policy_solver.set_gamma(0.9)

```

Listing 2: The method to change gamma

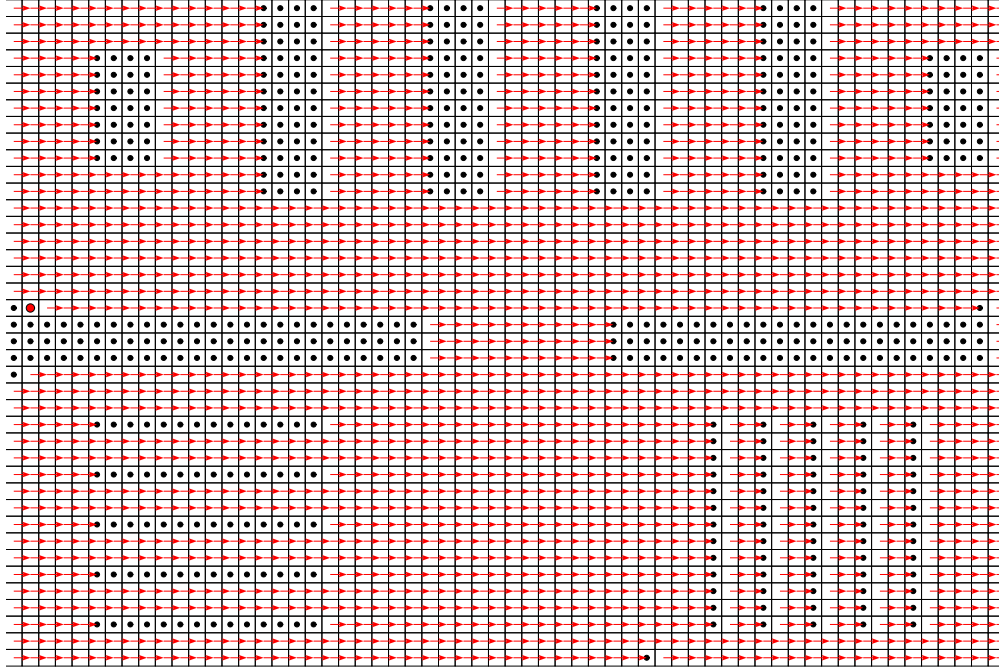


Figure 11: The initial policy

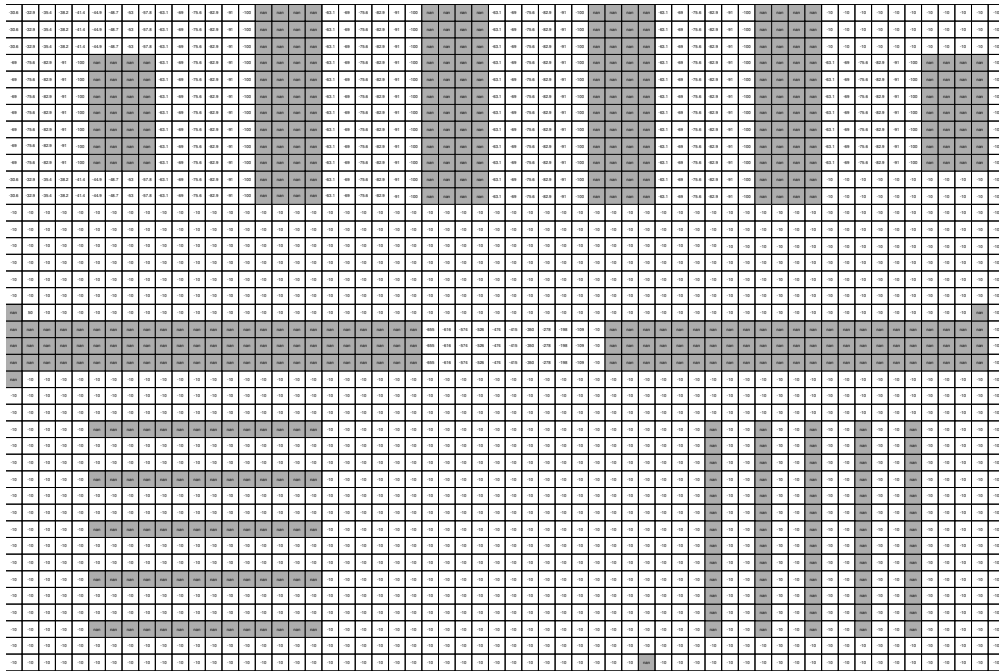


Figure 12: The value function after all the states have fully converged.

Q3(c)

To implement the policy improvement step, the `_improve_policy()` method was modified as follows:

1. Get the environment and map:

```
1     environment = self._environment
2     map = environment.map()
3
```

2. Initialize Policy Stability Flag:

```
1     policy_stable = True
2
```

3. Sweep systematically over all the states:

```
1         for x in range(map().width()):
2             for y in range(map().height()):
3
```

4. Ignore obstructed and terminal states:

- Obstruction cells and terminal states do not require policy updates, so they are skipped.

```
1         if map.cell(x, y).is_obstruction() or map.cell(x, y).is_terminal():
2             continue
3
```

5. Store the current policy action:

- This is used later to check whether the policy has changed.

```
1         old_action = self._pi.action(x, y)
2
```

6. Evaluate all possible actions:

- Initializes `best_action` (optimal action for this state).

```
1         best_action = None
2
```

- Sets `best_value` to negative infinity to ensure the first computed value will replace it.

```
1         best_value = float('-inf')
2
```

- 10 actions including special actions (terminate and none) from `low_level_actions.py`.

```
1         for a in range(10):
2
```

7. Compute the expected value $Q(s, a)$ for each action:

- Retrieves the **state transition probability distribution**:

- $s_{prime}(s')$: Possible next states.
- r : Corresponding rewards for each transition.
- p : Probabilities of transitioning to each next state.

```
1         s_prime, r, p = self._environment.next_state_and_reward_distribution
2         ((x, y), a)
```

- Computes the expected return for action a using **the Bellman Equation**:

$$Q(s, a) = \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')] \quad (10)$$

- $P(s'|s, a)$ - $p[t]$: Probability of transitioning to state s' .
- $R(s, a)$ - $r[t]$: Immediate reward for taking action a .
- γ - `self._gamma`: Discount factor.
- $V(s')$ - `self.v.value(...)`: Value of the next state s' .

```

1     new_value = 0
2     for t in range(len(p)):
3         sc = s_prime[t].coords()
4         new_value = new_value + p[t] * (r[t] + self._gamma * self._v.
5         value(sc[0], sc[1]))

```

8. Select the action with the highest Q-value:

```

1     if value > best_value:
2         best_value = value
3         best_action = a
4

```

9. Update the policy accordingly:

- Updates the policy to assign the best action for the current state (x,y)

```

1     self._pi.set_action(x, y, best_action)
2

```

10. Check if the policy has changed:

- If the new policy differs from the previous one, continue iterating.

```

1     if old_action != best_action:
2         policy_stable = False
3

```

11. Return policy stability status:

```

1     return policy_stable
2

```

The complete algorithm is shown below:

Algorithm 5 Policy Improvement Algorithm

```

1: function IMPROVEPOLICY
2:   environment  $\leftarrow$  self.environment                                ▷ Get the environment
3:   map  $\leftarrow$  environment.map()                                    ▷ Get the map
4:   policy_stable  $\leftarrow$  True                                       ▷ Initialize policy stability flag
5:
6:   for each x in map.width() do
7:     for each y in map.height() do
8:
9:       if map.cell(x,y).is_obstruction() or map.cell(x,y).is_terminal() then
10:        Continue                                                    ▷ Ignore obstructed and terminal states
11:      end if
12:
13:      old_action  $\leftarrow$  self.π.action(x,y)                        ▷ Store current policy action
14:      best_action  $\leftarrow$  None
15:      best_value  $\leftarrow$   $-\infty$ 
16:
17:      for each a in 0, 1, ..., 9 do
18:        s_prime, r, p  $\leftarrow$  environment.next_state_and_reward_distribution((x,y),a)
19:        new_value  $\leftarrow$  0                                          ▷ Initialize new value
20:
21:        for t = 0 to |p| do
22:          sc  $\leftarrow$  s_prime[t].coords()                            ▷ Get next state coordinates
23:          new_value  $\leftarrow$  new_value + p[t]  $\times$  (r[t] +  $\gamma \times$  self.v.value(sc[0], sc[1]))
24:        end for
25:
26:        if new_value > best_value then
27:          best_value  $\leftarrow$  new_value                                ▷ Update best value
28:          best_action  $\leftarrow$  a                                       ▷ Update best action
29:        end if
30:
31:      end for
32:
33:      self.π.set_action(x,y,best_action)                            ▷ Update policy
34:
35:      if old_action  $\neq$  best_action then
36:        policy_stable  $\leftarrow$  False                                ▷ Policy has changed
37:      end if
38:
39:    end for
40:  end for
41:
42:  return policy_stable                                              ▷ Return policy stability status
43: end function

```

Q3(d)

Modify the values in parentheses in this code to change the value of p :

```

1  test_probability = [
2      {"probability": 1},
3      {"probability": 0.9},
4      {"probability": 0.6},
5      {"probability": 0.3},
6  ]
7
8  for probability in test_probability:
9      airport_environment.set_nominal_direction_probability(probability["
10     probability"])

```

Listing 3: Creating a dictionary with different probability

When the discount factor $\gamma = 0.9$:

- After all iterations, the data has converged. However, as shown clearly in the two figures (Figure 8 & 9), the agent's optimal policy in the customs area at the bottom is highly irrational. It still follows its initial **strategy—continuously moving right** and even repeatedly colliding with obstacles.
- However, setting the discount factor to 1 would incur a massive computational burden. After extensive testing, we determined that 0.992 is the most appropriate value. Therefore, for all subsequent experiments, the discount factor will be set to 0.992.

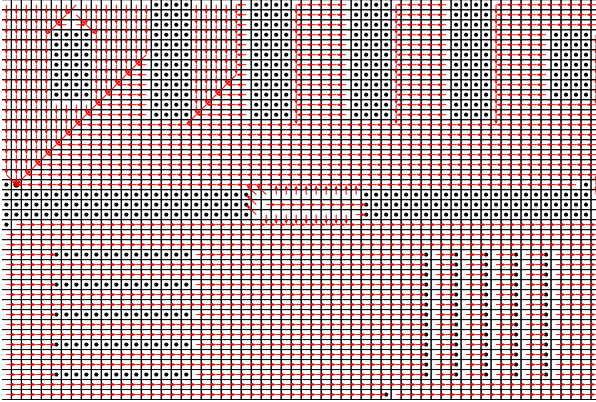


Figure 13: Policy when $\gamma = 0.9$

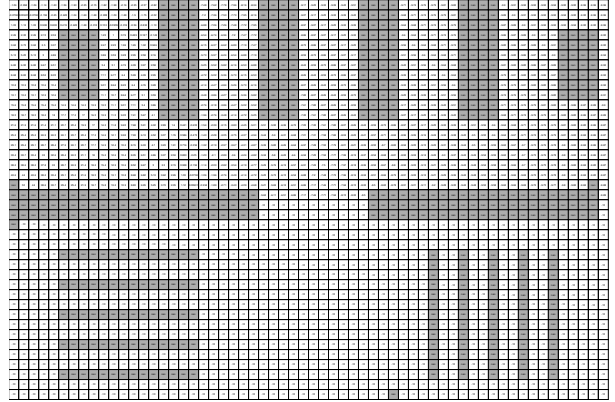
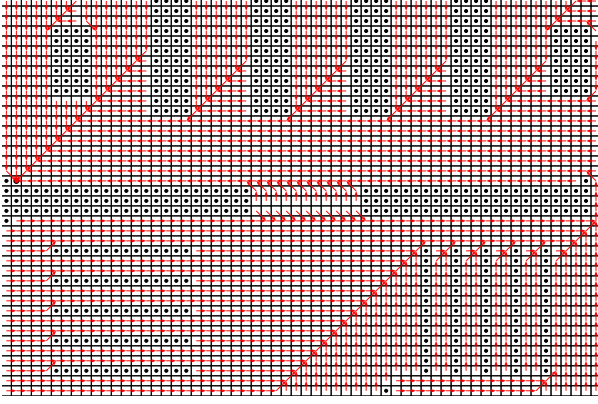
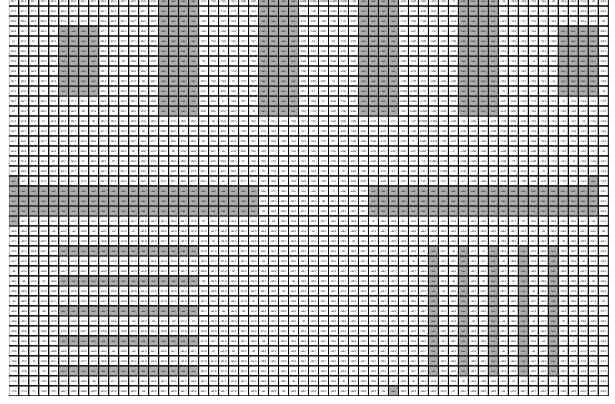


Figure 14: Value function when $\gamma = 0.9$

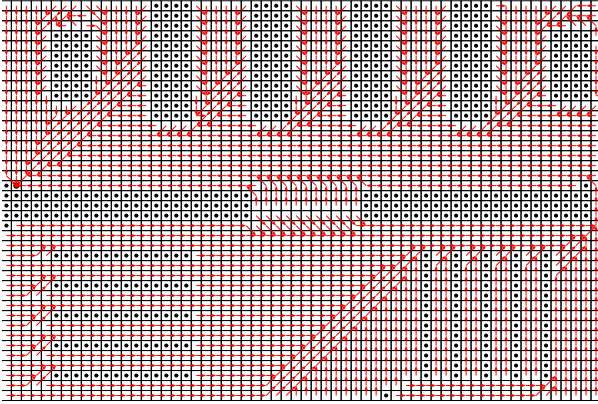
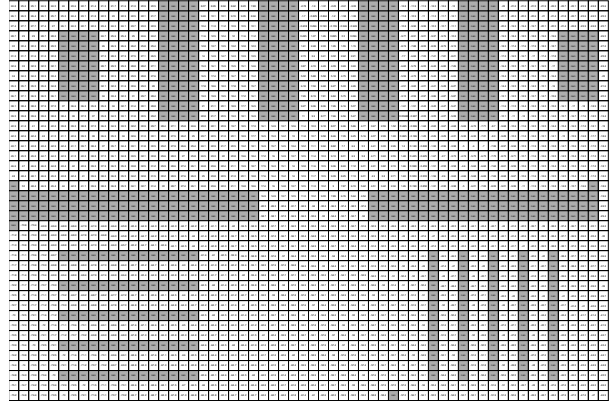
When the discount factor $\gamma = 0.992$:

- **p=1.0 (Deterministic)**
 - The **policy** is clear and well-defined. Every state has a shortest path to the end.
 - No deviation occurs as the robot moves exactly as commanded.
 - The **value function** for this case is the highest because the robot will **never** diminish its returns by colliding with obstacles or taking longer way.

Figure 15: Policy when $p=1$ Figure 16: Value function when $p=1$

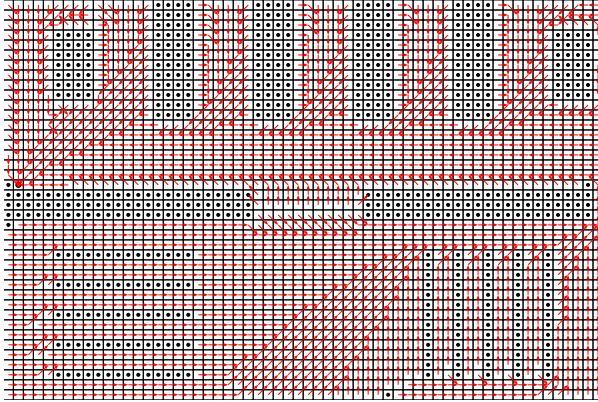
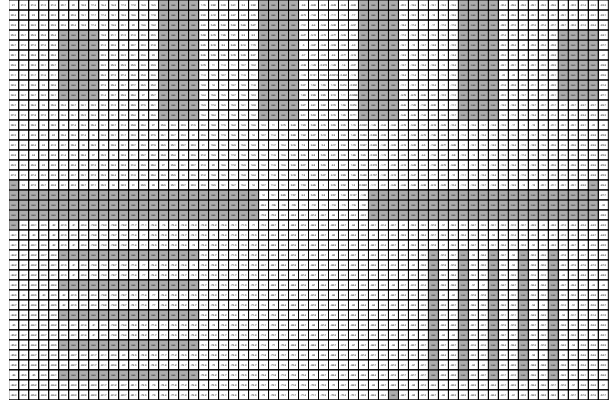
- **$p=0.9$ (Low uncertainty)**

- The **policy** remains mostly the same as in $p=1.0$, but minor deviations occur.
- The robot occasionally considers alternative routes in high-risk areas (beside luggage reclaim area).
- The **value function** for this case is a little lower than the previous case because the robot **might** diminish its returns by colliding with obstacles or taking longer way.

Figure 17: Policy when $p=0.9$ Figure 18: Value function when $p=0.9$

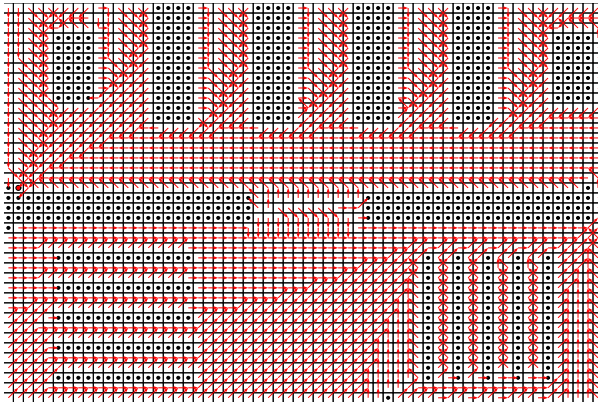
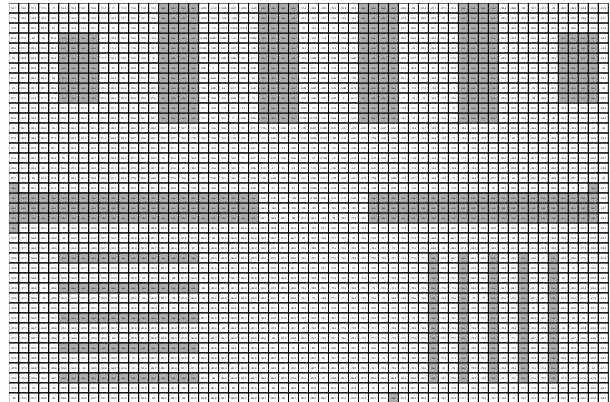
- **$p=0.6$ (Medium uncertainty)**

- The **policy** becomes more conservative, avoiding riskier paths.
- The robot takes longer routes to account for movement uncertainty.
- In high-cost areas (customs areas), robots will choose to crash into walls rather than make other executable moves.
- The **value function** for this case is low because the robot will **always** diminish its returns by colliding with obstacles or taking longer way.

Figure 19: Policy when $p=0.6$ Figure 20: Value function when $p=0.6$

- **$p=0.3$ (High uncertainty)**

- The **policy** is highly dispersed, showing no strict directionality.
- At this point the probability of the robot incorrectly executing a move has become higher than the expected move. So many of the robot's strategies have become unpredictable. It often does not choose the path that seems optimal.
- The **value function** for this case is very low, because the robot often opts for safer, highly redundant paths instead of the shortest path.

Figure 21: Policy when $p=0.3$ Figure 22: Value function when $p=0.3$

Summary:

- **For policy:** when $p \neq 1$, the robot may sometimes take a longer path instead of the shortest route due to several factors related to **policy randomness**, **expected reward optimization**.
 - **policy randomness:** In actual situation the robot does not always take the best action but instead has some probability of choosing a suboptimal or even worse action.
 - **expected reward optimization:** When the policy is probabilistic, the best strategy is not necessarily to take the shortest path but rather to maximize the expected cumulative reward. If a longer path offers a higher long-term reward, the robot might prefer it.
- **For value function:** As the probability of the robot performing the desired move decreases, the value function of the robot decreases and decreases.
 - This is due to the calculation through the Bellman Equation - the robot has a probability of choosing a sub-optimal route, resulting in a lower overall cumulative return.

- This could also lead to situations where the robot's final decision includes repeatedly colliding with obstacles and returning to its original position, resulting in an infinite loop. To mitigate this issue, an appropriate discount factor needs to be applied to adjust the value function, ensuring that the robot eventually learns to avoid such cycles. However, this correction significantly increases the computational complexity of the algorithm.

Q3(e)

There are two key parameters affecting the policy evaluation frequency:

- `max_policy_evaluation_steps_per_iteration`

This controls the numbers of iterations the policy evaluation runs before stopping.

- Higher values $\rightarrow$ More accurate value function but slower execution.
- Lower values $\rightarrow$ Faster execution but possibly suboptimal policies.

- `theta` (Convergence Threshold)

This determines when policy evaluation stops based on value function changes.

- Higher values (e.g., 0.1) $\rightarrow$ Faster but less accurate.
- Lower values (e.g., 1e-6) $\rightarrow$ Precise but slow.

Approach to investigate parameters:

1. Modify `q3_e.py` to evaluate different parameters.
2. Run multiple experiments varying `theta` and `max_policy_evaluation_steps_per_iteration`.
3. Measure execution time, number of iterations, and policy performance.
4. Analyze results to propose an optimal configuration.

Implementing the Investigation in `q3_e.py`:

- Set $p=0.8$:

```
1 airport_environment.set_nominal_direction_probability(0.8)
2
```

- Set discount factor value $\gamma = 0.992$:

```
1 policy_solver.set_gamma(0.992)
2
```

- Add timer to visualise performance more intuitively:

```
1 import time
2
3 # Start timing
4 start_time = time.time()
5
6 # Measure execution time
7 execution_time = time.time() - start_time
8
9 # Output results
10 print(f"Execution Time: {execution_time:.4f} seconds")
11
```

- Test different max evaluation steps (1,5,50,100) with the same theta value (1e-6):

```

1  # Experiment with different max_eval_steps when theta = 1e-6
2  test_configs = [
3      {"theta": 1e-6, "max_eval_steps": 1},
4      {"theta": 1e-6, "max_eval_steps": 5},
5      {"theta": 1e-6, "max_eval_steps": 50},
6      {"theta": 1e-6, "max_eval_steps": 100},]
7
8  for config in test_configs:
9      policy_solver.set_theta(config["theta"])
10     policy_solver.set_max_policy_evaluation_steps_per_iteration(config["
11     max_eval_steps"])

```

- Change the theta value and repeat the same process:

```

1  # Experiment with different max_eval_steps when theta = 0.01
2  test_configs = [
3      {"theta": 0.01, "max_eval_steps": 1},
4      {"theta": 0.01, "max_eval_steps": 5},
5      {"theta": 0.01, "max_eval_steps": 50},
6      {"theta": 0.01, "max_eval_steps": 100},]
7
8  # Experiment with different max_eval_steps when theta = 1
9  test_configs = [
10     {"theta": 1, "max_eval_steps": 1},
11     {"theta": 1, "max_eval_steps": 5},
12     {"theta": 1, "max_eval_steps": 50},
13     {"theta": 1, "max_eval_steps": 100},]
14

```

Analysis of performance:

- **max_eval_steps:(table 3)**
 - When theta keep constant. Increasing max_eval_steps from **1 to 5** significantly reduces the total number of policy iteration steps.
 - * This indicates that higher max_eval_steps allows for more accurate value function estimation, reducing the number of required policy iteration steps.
 - But further increasing max_eval_steps **to 50 or 100** provides diminishing returns.
 - * The total iteration steps increase. Theoretically, the more accurate the numerical evaluation, the fewer iterations it should take. The robot can find the optimal strategy more easily. But from the result of the experiments, we can find overly precise policy evaluation can lead to instability in policy improvement.
 - * By looking at the iterative process, I found that large max_eval_steps may cause the policy change drastically between iterations, leading to unnecessary fluctuations in the learning process.
 - * So, the total execution time and cost increases drastically.
- **theta:**
 - When the maximum number of evaluation steps is **smaller than 5**:
 - * The total number of policy iteration steps and execution times remains almost the same for all theta.
 - When the maximum number of evaluation steps become higher (**50 or 100**)
 - * When theta value gets low, the **execution time** decreases tremendously. This is even more obvious when the maximum number of evaluation steps is large.

Theta	Max Eval Steps	Total Policy Iteration Steps	Execution Time (s)
1e-06	1	124	9.4236
1e-06	5	71	7.5423
1e-06	50	77	25.8897
1e-06	100	88	35.2408
0.01	1	124	9.3026
0.01	5	71	7.4147
0.01	50	72	19.2253
0.01	100	89	27.0422
1	1	124	9.2155
1	5	70	7.0133
1	50	74	13.0380
1	100	75	13.8793

Table 6: Policy Iteration Performance for Different θ and Max Eval Steps

* However, the change in the number of **total iteration steps** is somewhat difficult to analyse, and there is no obvious trend or pattern of changing.

Max Eval Steps	Theta	Total Policy Iteration Steps	Execution Time (s)
1	1e-06	124	9.4236
1	0.01	124	9.3026
1	1	124	9.2155
5	1e-06	71	7.5423
5	0.01	71	7.4147
5	1	70	7.0133
50	1e-06	77	25.8897
50	0.01	72	19.2253
50	1	74	13.0380
100	1e-06	88	35.2408
100	0.01	89	27.0422
100	1	75	13.8793

Table 7: Policy Iteration Performance for Different Max Eval Steps and θ

In terms of performance, the optimal range for the maximum evaluation steps should be between 1 and 10. The value of **theta** has little impact on execution time, so we choose **0.01**. This ensures accuracy without increasing computational costs. After further testing, we generated a line graph showing the relationship between evaluation steps and execution time for values within this range (Figure 22). It is evident that `max_eval_steps = 3` is currently the best choice.

Accuracy Testing:

As mentioned earlier, a smaller theta and higher iteration steps indicate greater precision. To validate this, we compare an unlimited evaluation step setting with `theta = 1e-6` (Figure 24 and 25) against the currently optimal parameters: `theta = 0.01` and `max_eval_steps = 3` (Figure 26,27).

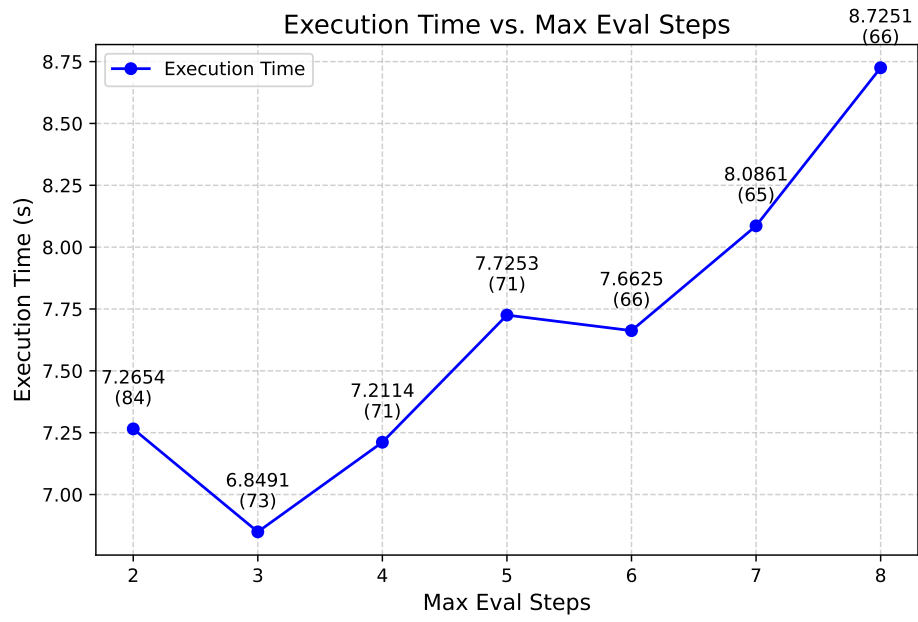
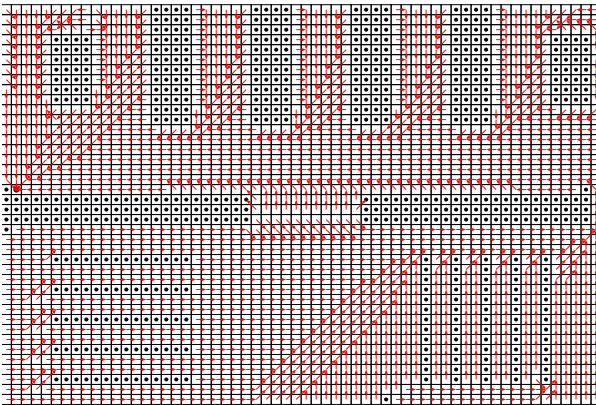
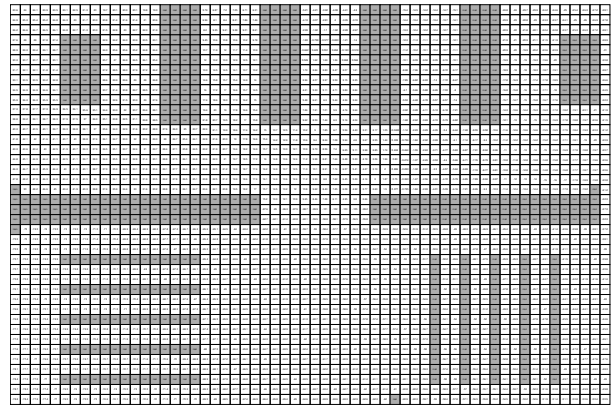
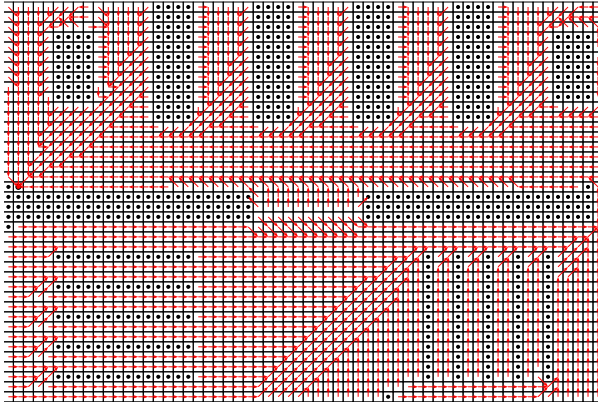
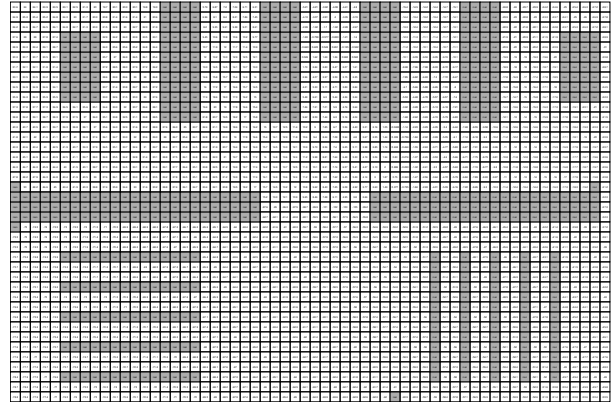


Figure 23: execution time vs max evaluation steps

Figure 24: Policy when $p=0.3$ Figure 25: Value function when $p=0.3$

Figure 26: Policy when $p=0.3$ Figure 27: Value function when $p=0.3$

Summary:

We find that the **value function** for this case is exactly the same. And there are only minor differences in the final **policy**. In real world, the robot's movement has randomness, and small sub-optimal decisions in policy have a negligible effect on the overall action.

So we decided that $\theta = 0.01$, $\text{max_eval_steps} = 3$ are the best parameters for this case.

Q3(f)

To implement the value iteration, there are two parts of the algorithm.

1. `_compute_optimal_value_function(self)`
2. `_extract_policy(self)`

The `_compute_optimal_value_function(self)` method is used to find and set the optimal value for each state:

1. Get the environment and map:

```
1 environment = self._environment
2 map = environment.map()
3
```

2. Initialize the iteration counter:

```
1 iteration = 0
2
```

3. Main Loop (While Loop):

The algorithm enters an infinite loop (while True:) that will continue until the value function converges or the maximum number of iterations is reached.

```
1 while True:
2
```

4. Sweep systematically over all the states:

```
1 for x in range(map().width()):
2     for y in range(map().height()):
3
```

5. Ignore obstructed and terminal states:

- Obstruction cells and terminal states do not require policy updates, so they are skipped.

```

1     if map.cell(x, y).is_obstruction() or map.cell(x, y).is_terminal():
2         continue
3

```

6. Store the current value function:

- This is used later to check whether the value has converged.

```

1     old_v = self._v.value(x, y)
2

```

7. Evaluate all possible actions:

- Sets `best_value` to negative infinity to ensure the first computed value will replace it.

```

1     best_value = float('-inf')
2

```

- 10 actions including special actions (terminate and none) from `low_level_actions.py`.

```

1     for a in range(10):
2

```

8. Compute the expected value $Q(s, a)$ for each action:

- Retrieves the **state transition probability distribution**:

- $s_{\text{prime}}(s')$: Possible next states.
- r : Corresponding rewards for each transition.
- p : Probabilities of transitioning to each next state.

```

1     s_prime, r, p = self._environment.next_state_and_reward_distribution
2         ((x, y), a)

```

- Computes the expected return for action a using **the Bellman Equation**:

$$Q(s, a) = \sum_{s'} P(s'|s, a) [R(s, a) + \gamma V(s')] \quad (11)$$

- $P(s'|s, a) - p[t]$: Probability of transitioning to state s' .
- $R(s, a) - r[t]$: Immediate reward for taking action a .
- $\gamma - \text{self._gamma}$: Discount factor.
- $V(s') - \text{self._v.value(...)}$: Value of the next state s' .

```

1     new_v = 0
2     for t in range(len(p)):
3         sc = s_prime[t].coords()
4         new_v = new_v + p[t] * (r[t] + self._gamma * self._v.value(sc
5         [0], sc[1]))

```

9. Select the action with the highest Q-value:

```

1     if new_v > best_value:
2         best_value = new_v
3

```

10. Update the policy accordingly:

- Updates the policy to assign the best action for the current state (x, y)

```

1     self._v.set_value(x, y, best_value)
2

```

11. Update the maximum deviation:

```

1     delta = max(delta, abs(old_v-best_value))
2

```

12. Check whether the conditions for terminating the evaluation iteration are met:

- If the maximum change across all states is below the threshold, we consider it converged:

```

1     if delta < self._theta:
2         break
3

```

- If the number of iterations exceeds the maximum limit (to prevent infinite loops at discount factor = 1):

```

1     if iteration >= self._max_optimal_value_function_iterations:
2         break
3

```

The `_extract_policy(self)` method is used to find and set the optimal policy for each state based on the optimal value function get from `_compute_optimal_value_function(self)` method:

```

1  def _extract_policy(self):
2
3      # Get the environment and map
4      print("extract policy")
5      environment = self._environment
6      map = environment.map()
7
8      # Sweep systematically over all the states
9      for x in range(map.width()):
10         for y in range(map.height()):
11
12             # Skip cells that are obstacles or terminal states.
13             if map.cell(x,y).is_obstruction() or map.cell(x,y).is_terminal():
14                 continue
15
16             cell = (x, y)
17
18             best_value = float('-inf')
19             best_action = None
20
21             # Iterate over all possible actions.
22             for a in range(10):
23
24                 # Compute p(s',r|s,a)
25                 s_prime, r, p = environment.next_state_and_reward_distribution
26
27                 (cell, a)
28
29                 new_v = 0
30
31                 # Sum over the rewards
32                 for t in range(len(p)):
33                     sc = s_prime[t].coords()
34                     new_v = new_v + p[t] * (r[t] + self._gamma * self._v.value
35                     (sc[0], sc[1]))
36
37                 # Keep track of the highest Q-value across actions.
38                 if new_v > best_value:
39                     best_value = new_v
40                     best_action = a
41
42                 # Update the state's value in the value function.
43                 # Set the new value in the value function
44                 self._pi.set_action(x,y,best_action)

```

Listing 4: method for extracting the policy

Algorithm 6 Value Iteration Algorithm

```

1: function COMPUTEOPTIMALVALUEFUNCTION
2:    $iteration \leftarrow 0$                                 ▷ Initialize iteration counter
3:   while True do
4:      $\delta \leftarrow 0.0$                                 ▷ Track maximum value change
5:
6:     for each state  $s \in \text{Environment}$  do
7:       if  $s$  is obstruction or terminal then
8:         continue                                        ▷ Skip irrelevant states
9:       end if
10:
11:       $v_{\text{old}} \leftarrow V[s]$                                 ▷ Store previous value
12:       $best\_value \leftarrow -\infty$ 
13:
14:      for each action  $a$  in possible actions do
15:        Compute expected value  $new\_v$  using transition probabilities
16:        if  $new\_v > best\_value$  then
17:           $best\_value \leftarrow new\_v$ 
18:        end if
19:      end for
20:
21:       $V[s] \leftarrow best\_value$ 
22:       $\delta \leftarrow \max(\delta, |v_{\text{old}} - best\_value|)$ 
23:      Print state update
24:    end for
25:
26:     $iteration \leftarrow iteration + 1$ 
27:    if  $\delta < \theta$  or  $iteration \geq \text{max iterations}$  then
28:      break                                              ▷ Convergence condition met
29:    end if
30:
31:  end while
32:  Print total iterations
33: end function
34:
35:
36: function EXTRACTPOLICY
37:   for each state  $s$  in Environment do
38:     if  $s$  is obstruction or terminal then
39:       continue                                        ▷ Skip irrelevant states
40:     end if
41:
42:      $best\_action \leftarrow \text{None}$                             ▷ Initialize best action
43:
44:     for each action  $a$  in possible actions do
45:       Compute action value using transition probabilities
46:        $best\_action \leftarrow \arg \max action\_values$ 
47:     end for
48:
49:      $\pi[s] \leftarrow best\_action$                             ▷ Update the policy
50:   end for
51: end function

```

Q3(g)

In order to compare policy iteration and value iteration, we modify q3_g.py as following:

- Create a function for creating two instances: Policy Iterator & Value Iterator:

```

1  def solver(type, gamma):
2      """
3      type: Type of the iterator
4      gamma: Discount factor
5      """
6
7      # Create the valuevalue iterator instance
8      if type == "value":
9          policy_solver = ValueIterator(airport_environment)
10         policy_solver.initialize()
11         print(f"Value iteration:")
12
13     # Create the policy iterator instance
14     else:
15         policy_solver = PolicyIterator(airport_environment)
16         policy_solver.initialize()
17         policy_solver.set_max_policy_evaluation_steps_per_iteration(3)
18         print(f"Policy iteration:")
19
20     # Set the same gamma value for two instances
21     policy_solver.set_gamma(gamma)
22
23     # Set the same theta value for two instances
24     policy_solver.set_theta(0.01)

```

Listing 5: q3_g.py The solver function

- Calling the function with different argument: (Since we found that when the value of gamma is less than 0.99, the final policy will have a very high number of suboptimal decisions. So we do not analyse these values)

```

1  for gamma in (1, 0.999, 0.992, 0.99):
2      print(f"gamma = {gamma}")
3      solver("value", gamma)
4      solver("policy", gamma)
5

```

Listing 6: q3_g.py Calling the function

Analysis of performance:

Whether or not value functions and policies are drawn can greatly affect execution time. So we analyse it in two cases

- **Without drawing results (Table 9):**
 - **Value Iteration computational cost increases with decreasing γ .**
 - * Execution time increases from **8.1188s** ($\gamma = 1.0$) to **34.8380s** ($\gamma = 0.99$).
 - * The number of iterations increases from **109 to 461**, showing slower convergence for smaller γ .
 - **Policy Iteration is more stable.**
 - * Execution time remains stable (**6.9–7.3s**) across different γ values.
 - * The number of steps is almost constant (**73–75**), showing less sensitivity to γ .
 - **In this case, policy Iteration is better. Especially if the value of γ is small (0.9).**

- * The execution time is relatively shorter and results can be produced more quickly.
- * While it is not able to accurately derive the exact computational cost, we can deduce from the total number of iterations and runtime that the policy iterations are cheaper.

Gamma	Value Iteration Steps	Value Iteration Time (s)	Policy Iteration Steps	Policy Iteration Time (s)
1.000	109	8.1188	74	7.2602
0.999	111	7.9909	75	7.0986
0.992	228	16.1394	73	6.9030
0.99	461	34.8380	73	6.8969

Table 8: Comparison of Value Iteration and Policy Iteration for Different γ Values (doesn't draw the result)

- With drawing the results (Table 8):
 - **Drawing results increases execution time significantly**
 - * For $\gamma = 1.0$: Value Iteration time increases from **8.1188s** to **28.1105s (+246%)**.
 - * Policy Iteration is even more affected, increasing from **7.2602s** to **34.9965s (+382%)**.
 - **Value Iteration still requires more steps**
 - * The number of steps remains the same whether results are drawn or not.
 - * Value Iteration requires significantly more steps for smaller γ , increasing execution time.
 - **Policy Iteration Becomes Less Efficient for Larger γ**
 - * Execution time increases sharply from **7.26s** to **34.99s** for $\gamma = 1.0$.
 - * This suggests Policy Iteration is more affected by visualization overhead than Value Iteration. This is because the policy iteration updates the policy table at each iteration, whereas the value iteration updates the policy table only once at the end.
 - **In this case, best method based on the value of γ .**
 - * When γ is very close to 1, Value Iteration will get the result more quickly. And since there is no need to update the policy table all the time, the computational cost is relatively low.
 - * When γ is smaller (0.99), Policy Iterations show better performance.

Gamma	Value Iteration Steps	Value Iteration Time (s)	Policy Iteration Steps	Policy Iteration Time (s)
1.000	109	28.1105	74	34.9965
0.999	111	28.2900	75	41.2164
0.992	228	53.6302	73	38.9859
0.99	461	107.9781	73	35.5969

Table 9: Comparison of Value Iteration and Policy Iteration for Different γ Values (draw the result)

- **Summary**

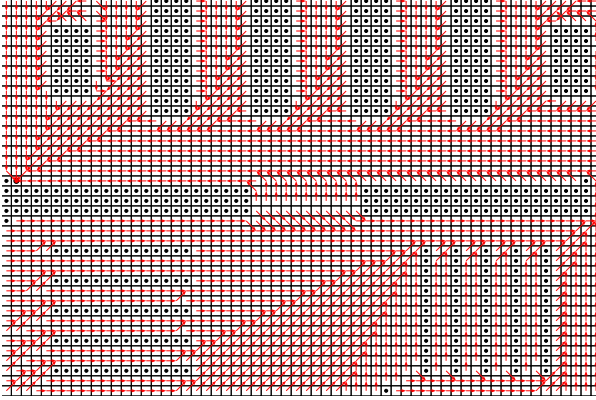
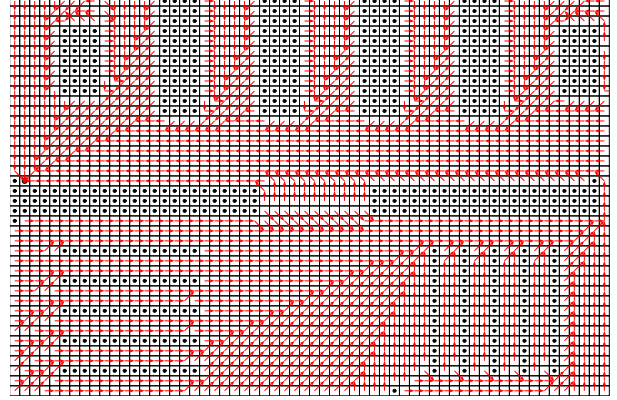
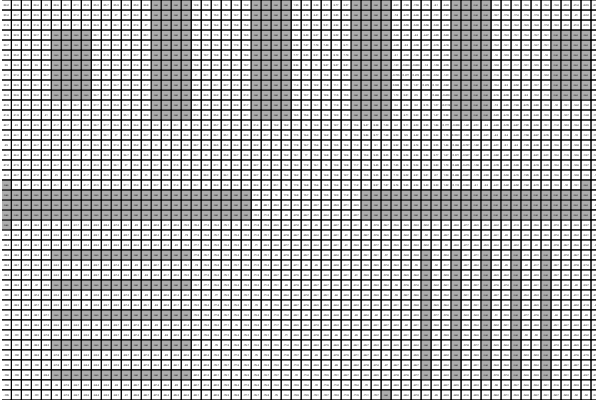
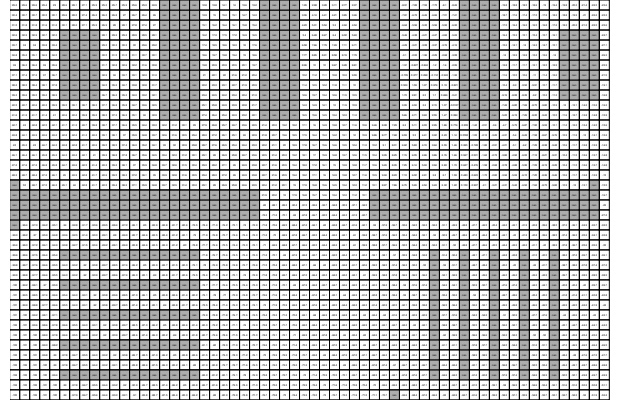
Condition	Best Algorithm
Without Drawing Results	Policy Iteration performs better for all γ .
With Drawing Results:	
γ very close to 1	Value Iteration is faster, as it avoids frequent policy updates.
$\gamma = 0.99$ or lower	Policy Iteration performs better, requiring fewer iterations.

Table 10: Performance Summary of Value Iteration and Policy Iteration

Accuracy test:

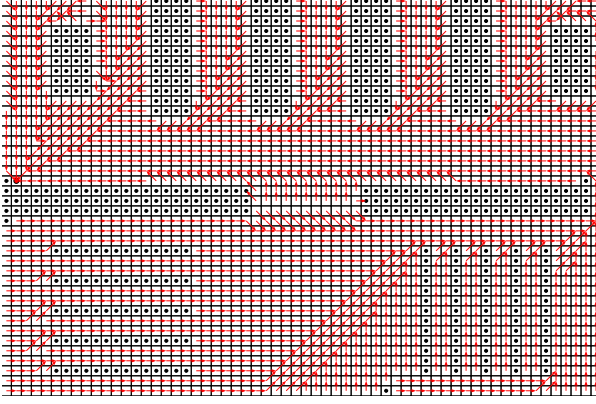
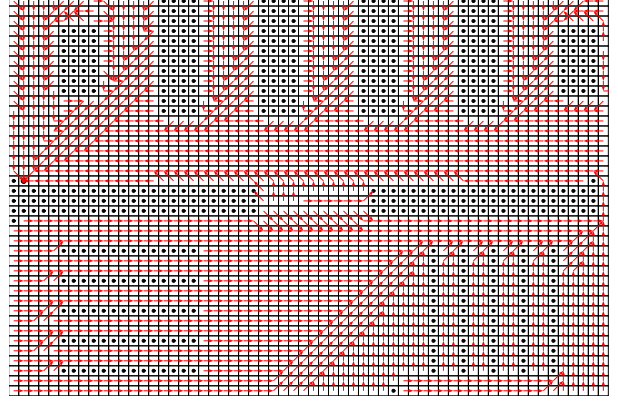
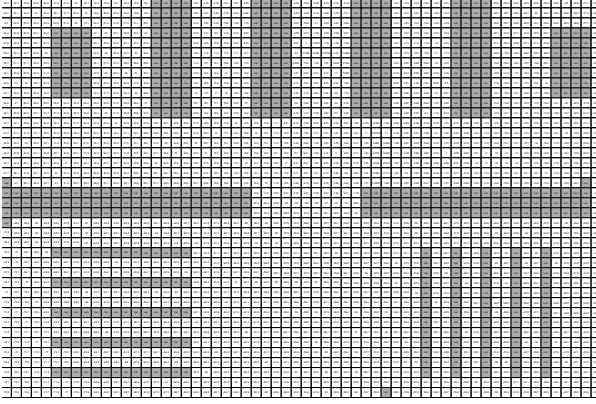
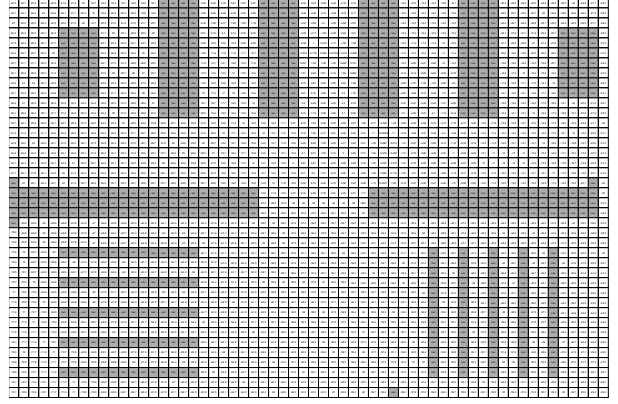
- **When γ close to 1:**

- The **policy** (Figure 30,31) of two methods is almost the same. Only some decisions beside the luggage reclaim area have some differences. But neither appeared to be an obviously unreasonable route.
- Some states will have very small differences (smaller than 0.1) in **values** (Figure 32,33). This is due to errors caused by different iteration methods and does not affect the policy decisions.

Figure 28: Policy by policy iteration $\gamma = 1$ Figure 29: Policy by value iteration $\gamma = 1$ Figure 30: Value function by policy iteration $\gamma = 1$ Figure 31: Value function by value iteration $\gamma = 1$

- **When γ is small (0.99):**

- The policy is similar to the case of $\gamma = 1$, most of the decisions are the same and only a small number are different.
- However, in the high cost region (customs region), we find that the policy derived from the **value iteration** is not reasonable. The cost per move here is very high, yet the robot chooses to move to the right in this area rather than leave the area immediately. This is because the robot ended the iteration early in the value iteration process. The result is that no improvements were made to this part of the robot's strategy, and it still uses the initial strategy.
- After comparing the **value equations** for the customs region, we find that these values get from value iteration do not reach the ones get from policy iteration. This proves that value iteration may not fully converge before extracting the policy, leading to slight inaccuracies.

Figure 32: Policy by policy iteration $\gamma = 0.99$ Figure 33: Policy by value iteration $\gamma = 0.99$ Figure 34: Value function by policy iteration $\gamma = 0.99$ Figure 35: Value function by value iteration $\gamma = 0.99$ **Summary:**

Algorithm	Accuracy	Efficiency
Value Iteration	May have slight value function inaccuracies due to early policy extraction.	Computationally efficient per iteration but may require many steps.
Policy Iteration	More accurate as it fully evaluates policies before updating.	Computationally expensive per iteration but requires fewer total steps.

Table 11: Comparison of Value Iteration and Policy Iteration in Terms of Accuracy and Efficiency

- In this current situation, I think **policy iteration** is the more appropriate approach. In practice, the robot does not need to draw a visualisation of the policy. In this case, policy iteration can be compatible with both accuracy and efficiency.
- However, if the robot needs to plot the resultant policy frequently, **value iteration** is better. Because in this case, the robot can produce results faster and the computational cost is relatively low.

Q4

Components of FMDP

State: The robot's current location, battery level, the next cleaning site to visit, etc.

- **Location:** Discrete grid coordinates.
- **Battery capacity:** Discrete grades (e.g. 0%-20%, 20%-50%, 50%-100%).
- **Next cleaning site:** Discrete target points.

Action: The robot can choose to go directly to the next cleaning site or to a charging station to recharge. If the battery level falls below a certain threshold, the robot can only choose to go to the charging station.

Transition Probability: The probability that a robot moves from one state to another after performing an action. For example, when a robot chooses to go to the next cleaning site, it may successfully arrive, or it may need to be rescued because its battery is exhausted. If the robot chooses to charge, it will be charged with a certain amount of electricity after a certain time.

If the robot chooses to go to the next cleaning site, the probability of successful arrival is p and the probability of battery exhaustion is $1 - p$.

Reward Function:

- **Reward for trash/toilet cleaning:** Positive fixed value.
- **Reward for arriving at the charging station and charging:** A function based on the amount of charge $R_{\text{charge}}(b)$ where b is the amount of charge.
- **Battery depletion penalty:** Large fixed negative value (indicating high rescue costs).
- **Cost of movement:** Can be negative, indicating energy consumption.

Discount Factor (γ): Generally set at $0 < \gamma \leq 1$. If $\gamma = 1$, it indicates that future rewards are weighted equally with current rewards. A smaller γ makes the algorithm favor short-term rewards, whereas a larger γ makes it favor long-term rewards.

The Application of Policy Iteration and Value Iteration

Policy Iteration: Policy iteration includes two steps: **policy evaluation** and **policy improvement**. First, the value function of the current policy is evaluated, and then the policy is improved based on the value function. This process is repeated until the policy converges to the optimal policy.

In practice, we assume the current strategy π , calculate the value $V(s)$ for each state, then update π based on $V(s)$, making the new policy more profitable. This step is repeated until convergence.

For each s , select the action a that maximizes:

$$\sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

Value Iteration: Value iteration approximates the optimal value function by constantly updating the value function and then deriving the optimal strategy according to the optimal value function.

We update $V(s)$ by directly iterating the Bellman equation and finally select the optimal action through the greedy strategy:

$$V(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

Discussion: Policy Iteration vs. Value Iteration

- **Policy iteration:** typically requires more computing resources but converges to the optimal policy more quickly.
- **Value iteration:** approximates the optimal strategy by constantly updating the value function, making it more suitable for problems with large state spaces.

- Policy iteration has a fast convergence speed but high computational cost per policy evaluation.
- Value iteration is computationally efficient per step but may require more iterations to converge.